Towards Out-of-Core Simulators for Quantum Computing

Immanuel Trummer
Cornell University
Ithaca, NY, USA

ACM Reference Format:
Immanuel Trummer. 2024. Towards Out-of-Core Simulators for Quantum Computing. In *Workshop on Quantum Computing and Quantum-Inspired Technology for Data-Intensive Systems and Applications (Q-Data '24), June 09–15, 2024, Santiago, AA, Chile.* ACM, New York, NY, USA, 1 page. https://doi.org/10.1145/3665225.3665441

Introduction. The simulation of quantum computers on classical machines informs the development of future quantum hardware devices as well as the development of quantum algorithms. While a variety of simulation packages are nowadays available, their scope is limited to systems with a small number of qubits. Typically, the primary bottleneck is the amount of memory required to simulate quantum computations with large numbers of qubits exactly. To fully capture the state of a quantum system, the amount of required memory grows exponentially in the number of simulated qubits. As most simulators[4, 5] target in-memory computation, this places severe constraints on the size of quantum systems that can be simulated on commodity machines.

Over the past decades, the database community has developed various algorithms and systems that aim at making out-of-core computation efficient. This work assumes, mostly, that data are far too large to fit into main memory. Applying such techniques to the domain of quantum simulation could significantly expand the size of the systems that can be simulated. The most straightforward way to do so is by mapping operations for simulating quantum circuits to languages supported by systems for out-of-core computation. The most prominent among them is the SQL language. The following sections show how to represent quantum states as SQL tables and quantum operations as SQL queries. Future work will aim at implementing and evaluating the proposed approach.

Background. The following sections refer to the Schrödinger algorithm, a popular approach for simulating quantum computation used by many simulators [4, 5]. It simulates quantum circuits by applying each gate consecutively to a quantum state, represented as a vector of amplitudes. Different from alternative approaches such as the one by Feynman [1], it scales to deep quantum circuits. Different from other recent work [2], focusing on sparse vectors by adding columns to represent tensor indexes explicitly, the approach proposed in the next sections is optimized for dense vectors (even though the database management system may exploit sparseness by using corresponding column compression methods internally).

Representing States. The state of a quantum system can be represented by amplitudes for each combination of qubit values. Each amplitude is a complex number. SQL does not support complex numbers directly but real and imaginary parts can be stored as float values. The following table associates qubit values, stored as strings, with amplitudes.

```
CREATE TABLE States(qubits text, areal float, aimg float);
```

Implementing Gates. A gate of a quantum circuit that applies to a single qubit can be represented by a unitary matrix with complex entries `qij` where both `i` and `j` are indexes between one and two. After applying the gate to qubit `k`, the new amplitude for a state `s` can be calculated from the prior amplitudes of states `s1` and `s2` where `s1` and `s2` only differ in the value for qubit `k` and have equal values to `s` in all other qubits (prior work provides details [4]). Assuming real-valued entries `qij`, the new amplitudes are calculated by an SQL query that matches states differing only in the `k`-th qubit for a system with `n` qubits. The `UNION ALL` statement collects the new amplitudes for states `s1` and `s2`, differing only in their value for the `k`-th qubit.

```
WITH (SELECT A.quibits as Aqubits, B.qubits as Bqubits,
 A.areal as Aareal, A.aimg as Aaimg,
 B.areal as Bareal, B.aimg as Baimg
 FROM States A, States B WHERE
 SUBSTRING(A.qubits,1,k-1)=SUBSTRING(A.qubits,1,k-1)
 AND SUBSTRING(A.qubits,k+1,n-k)=SUBSTRING(A.qubits,k+1,n-k)
 AND A.qubits < B.qubits) AS J
(SELECT Aqubits, q11*Aareal+q12*Bareal, q11*Aaimg+q12*Baimg)
UNION ALL (SELECT Bqubits, q21*Aareal+q22*Bareal,
 q21*Aaimg+q22*Baimg)
```

The query can be generalized to handle complex entries `qij` as well (modeling the multiplication of complex numbers as an SQL function [2]). Gates that operate on pairs of qubits, using one qubit as control qubit, can be handled by introducing SQL `CASE` statements (applying a function only if the control qubit is set to one). The supported gates are universal for quantum computation [3].

REFERENCES

[1] Ethan Bernstein and Umesh Vaziranit. 1993. Quantum complexity theory. In *Proceedings of the Annual ACM Symposium on Theory of Computing*, Vol. Part F1295. 11–20. https://doi.org/10.1145/167088.167097
[2] Mark Blacher, Julien Klaus, Christoph Staudt, Sören Laue, Viktor Leis, and Joachim Giesen. 2023. Efficient and Portable Einstein Summation in SQL. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–19. https://doi.org/10.1145/3589266
[3] David P. DiVincenzo. 1995. Two-bit gates are universal for quantum computation. *Physical Review A* 51, 2 (1995), 1015–1022. https://doi.org/10.1103/PhysRevA.51.1015 arXiv:9407022 [cond-mat]
[4] Mikhail Smelyanskiy, Nicolas P. D. Sawaya, and Alán Aspuru-Guzik. 2016. qHiPSTER: The Quantum High Performance Software Testing Environment. (2016), 1–9. arXiv:1601.07195 http://arxiv.org/abs/1601.07195
[5] Xin Chuan Wu, Sheng Di, Emma Maitreyee Dasgupta, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic T Chong. 2019. Full-state quantum circuit simulation by using data compression. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC.* https://doi.org/10.1145/3295500.3356155 arXiv:1911.04034

Q-Data '24, June 09–15, 2024, Santiago, AA, Chile

ACM ISBN 979-8-4007-0553-3/24/06
https://doi.org/10.1145/3665225.3665441